



REYKJAVÍK UNIVERSITY

PyPI-SCADA

Python Package Index: Supply Chain Attack Detection Analysis
Situation Report I

Author:

Alexander Máni Einarsson - alexanderme22@ru.is

Project Supervisor

Dr. Jacqueline Clare Mallett - jacky@ru.is

Examiner:

Kristrún Lilja Júlíusdóttir - kristrun.lilja.juliusdottir@orkuveitan.is

February 19, 2026

Table of Contents

1	Project Description	1
1.1	Background	1
1.2	Goals	1
1.3	Tech Stack	1
2	Methodology and Work Procedure	3
2.1	Work Method and Organization	3
2.2	Measurements	3
3	Project Management and Timeline	4
4	Risk Analysis	5
5	Requirements and Design	6
5.1	System Architecture	6
5.1.1	PyPI Injector	6
5.1.2	PyPI Basic Simulator	7
5.1.3	PyPI Detection Analyzer	8
6	Progress and Time Tracking	9
6.1	Completed Tasks	9
6.2	Worked Hours	11

1 Project Description

1.1 Background

The software supply chain is a critical vector for cyberattacks. As the central repository for the Python ecosystem, the Python Package Index (PyPI) manages over 730,000 packages, and serves trillions of downloads annually [1]. Recent security incidents, including Department of Justice subpoenas regarding PyPI user data, highlight the platform’s role as a high-value target for state-sponsored and criminal actors [2]. Vulnerabilities in dependency chains can propagate downstream, affecting global infrastructure utilized by organizations such as NASA and IBM [3].

This project, conducted at the **Frostbyte Cybersecurity Research Laboratory** at Reykjavik University, evaluates the efficacy of Large Language Models (LLMs) compared to traditional Static Application Security Testing (SAST) tools in detecting malicious code uploads.

1.2 Goals

The project aims to construct a secure automated pipeline to:

- **Simulate Repository Environments:** Operate a PEP 503 compliant mirror to host and test malicious packages safely.
- **Perform Differential Analysis:** Implement a code diff engine to isolate logical changes in package updates.
- **Benchmark Detection Efficacy:** Compare LLMs (e.g., Gemini, GPT) against SAST tools (Bandit, Semgrep) using four primary attack vectors: SilentSync (Trojan persistence), Typosquatting (Colorama), Credential Takeover, and Dependency Confusion (Torchtriton).

1.3 Tech Stack

The system is developed using a Python-centric architecture designed for modularity and safety.

- **Core Language:** Python 3.9+ for all logic components.
- **Web Framework:** Flask (Python) to serve the PEP 503 compliant simulator index.
- **Automation:** Bash/Shell scripts utilizing `twine` and `pip` for package injection and verification
- **Data Management:** SQLite (`sqlite3`) for metadata storage and result persistence.
- **Configuration:** YAML for prompt templates and standardized detection settings.

The system utilizes a multi-layered detection architecture as shown in Table 1.

Table 1: Detection Methodology Tech Stack

Traditional SAST	Proprietary LLMs	Open Source LLMs
Bandit (AST Linter)	Google Gemini 1.5	Meta Llama 3 (via Groq)
Semgrep (Pattern Matching)	OpenAI GPT-4o	Mistral Large
Heuristic Static Checks	Anthropic Claude 3.5	DeepSeek-V3

2 Methodology and Work Procedure

2.1 Work Method and Organization

The research follows an iterative **Scrum-based methodology**. Development is compartmentalized into isolated repositories for the Simulator, Injector, and Analyzer.

- **Isolation:** Work with malware will be conducted on sanitized isolated VM on a safe infrastructure network within Frostbyte, decided with lab system administrators.
- **Supervision:** Bi-weekly meetings with Dr. Jacky Mallett ensure alignment with academic standards and safety protocols.

2.2 Measurements

To quantify detection performance, the following metrics are utilized:

- **Detection Accuracy:** Percentage of correctly identified malicious samples [2].
- **False Positive Rate (FPR):** Current tools often generate FPRs between 15% and 30% [4].
- **Analysis Latency:** Target analysis time for static components is $< 0.5s$.
- **Reasoning Quality:** Qualitative evaluation of LLM-generated explanations for maliciousness.
- **Prompt Quality:** What prompt was used according to what strategy.

3 Project Management and Timeline

The following timeline outlines the 400-hour development and research cycle for the **PyPI-SCADA** project.

- **Phase I: Research, Design, and Setup (80 Hours)**
 - **Orientation (Jan 12–25):** Infrastructure provisioning on Frostbyte VM, attack vector research (SilentSync, Typosquatting, etc.), and tool selection.
 - **Architectural Design (Jan 26–Feb 8):** System design for simulator/analyzer and formal Risk Analysis.
 - **Status Meeting 1 (Feb 9–15):** Initial presentation and project plan validation.
- **Phase II: Core Engine Development (100 Hours)**
 - **Simulator & Storage (Feb 16–Mar 1):** Implementing the PEP 503 compliant repo and metadata database.
 - **Diffing Engine (Mar 2–8):** Logic for extracting modifications between package versions.
 - **Status Meeting 2 (Mar 9–15):** Technical demonstration of the prototype engine.
- **Phase III: Integration and Benchmarking (100 Hours)**
 - **Pipeline Integration (Mar 16–Apr 5):** Integrating SAST tools (Bandit, Semgrep) and LLM APIs (Gemini, GPT) with custom prompt engineering.
 - **Dataset Preparation (Apr 6–12):** Synthesis of malicious samples and benign control groups.
 - **Execution Period (Apr 13–26):** Automated benchmarking suite execution during university exam window.
- **Phase IV: Analysis and Final Thesis (100 Hours)**
 - **Evaluation (Apr 27–May 3):** Status Meeting 3 (Dress Rehearsal) and result comparison (Detection vs. False Positives).
 - **Final Documentation (May 4–17):** Conclusion writing, code polishing, and submission to Skemman.
 - **Final Presentation (May 18–22):** Public defense and project wrap-up.

4 Risk Analysis

Risk is quantified as $Risk = Probability \times Impact$. Here 1 is the minimum and 5 is the highest for probability and impact, making factor the factor 1-25.

Risk	Prob.	Imp.	Factor	Mitigation Strategy
Malware Execution	1	5	5	Use of isolated VMs.
Development Environment Weak to Supply Chain Attacks	1	5	5	Use of fixed versions of python packages that are deemed safe.
Scope Creep	3	4	12	Refactor Tasks to Fit a Laboratory proof of concept
LLM API Costs	4	2	8	Utilization of project funding and Groq-hosted open models.
False Positives Undermining Evaluation Validity	3	4	12	Benchmark against curated benign datasets and manually validate edge cases.
Reproducibility Risk of Experimental Results	2	4	8	Store prompts, configurations, and hashes of analyzed packages to ensure replayability.

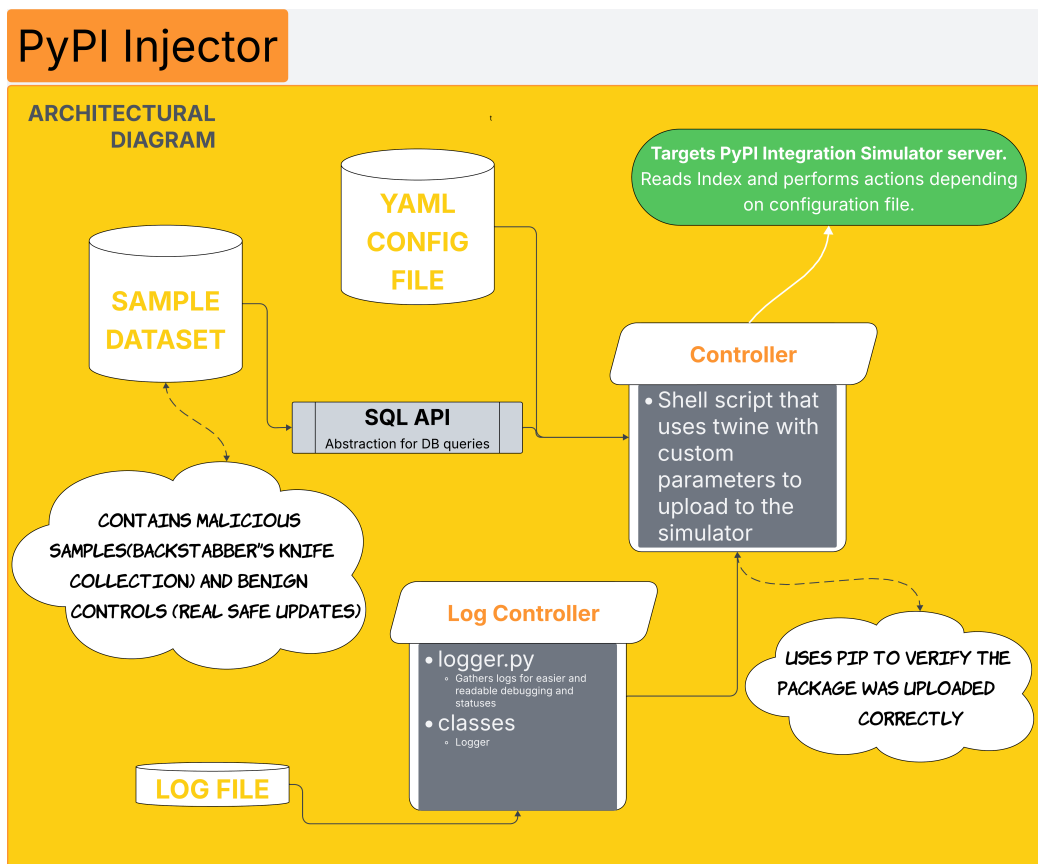
5 Requirements and Design

5.1 System Architecture

5.1.1 PyPI Injector

The Injector automates the upload of test data. It includes:

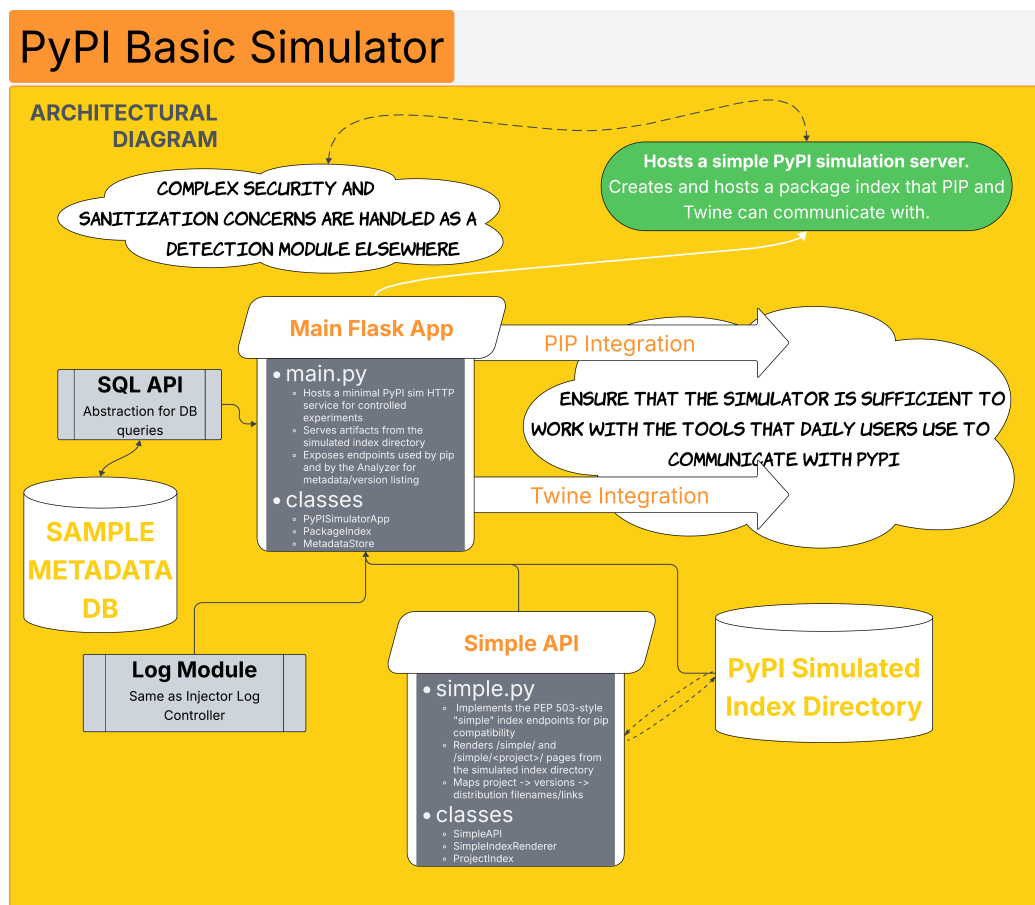
- **Controller:** A shell script utilizing `twine` to target the simulation server.
- **sql.py:** A SQL API abstraction for serving samples from the *Sample Dataset* (e.g., SilentSync, Colorama).
- **YAML Config:** Defines standard configurations and prompt templates for different detection methods.



5.1.2 PyPI Basic Simulator

A minimal PEP 503 compliant environment:

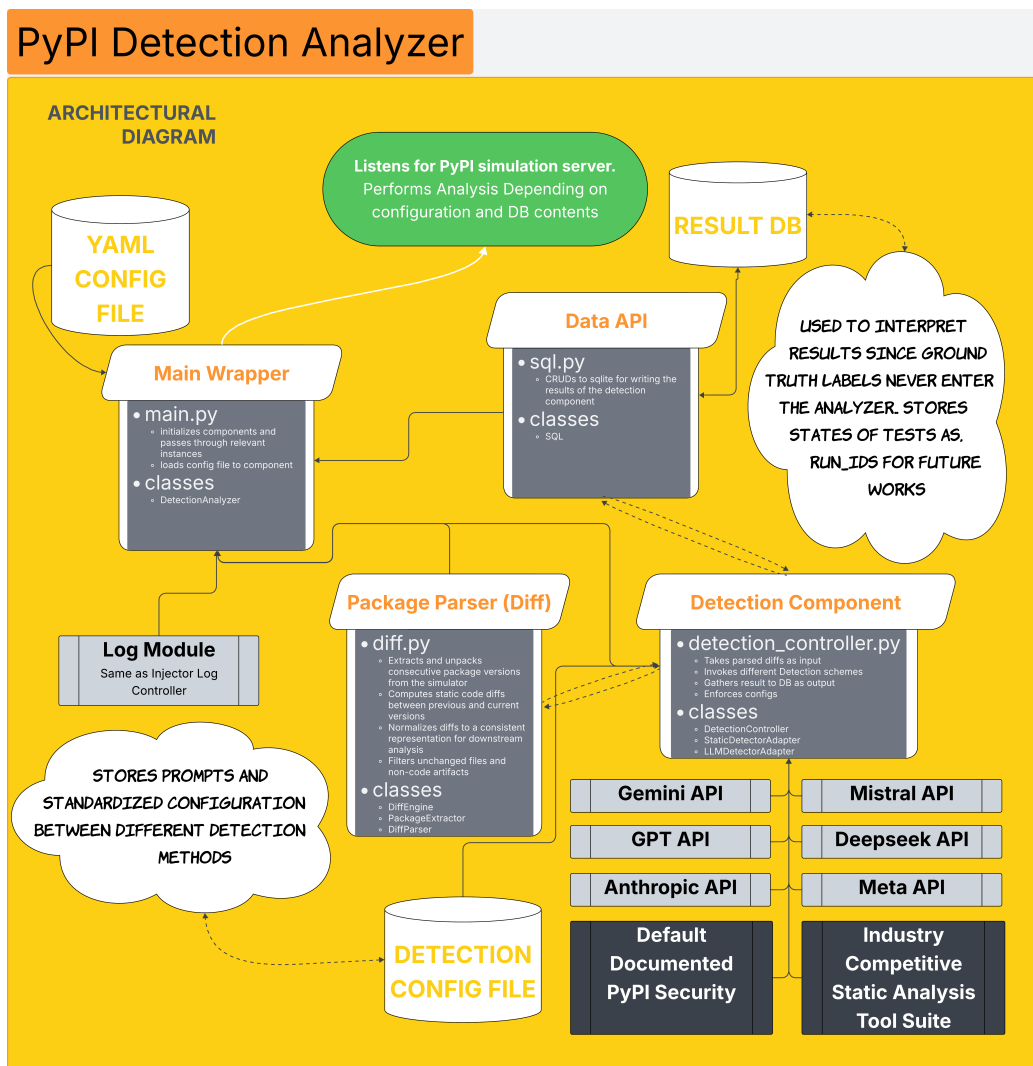
- **main.py:** A Flask-based application serving the PyPI simulation index.
- **simple.py:** Implements the `/simple/<project>/` endpoints for pip compatibility.
- **SQL API:** Stores metadata for simulated versions.



5.1.3 PyPI Detection Analyzer

The core engine for differential analysis:

- **diff.py (Package Parser):** Computes static code diffs between consecutive package versions to isolate new code logic [5].
- **detection_controller.py:** Orchestrates the StaticDetectorAdapter and LLMDetectorAdapter to invoke relevant APIs and tools (Gemini, GPT, Bandit, etc.).
- **Result DB:** Persists RUN_IDS and analysis states for longitudinal performance tracking.



6 Progress and Time Tracking

The project fulfills the 12 ECTS requirement (approx. 300–360 man-hours).

Phase	Focus	Planned (h)
Phase 1	Lab Induction & Research	80
Phase 2	Simulator & Diff Engine Development	100
Phase 3	SAST & LLM Adapter Integration	100
Phase 4	Benchmarking & Final Thesis	100
Total		380

6.1 Completed Tasks

The following items have been successfully finalized during the initial research and setup period.

Category	Task Description	Status
Dataset	Dataset acquisition: Full access to the Backstabber’s Knife Collection secured via Dr. Ohm (University of Bonn).	Completed
Infrastructure	Provisioned virtual machine on Frostbyte infrastructure.	Completed
Tools	Initialized Git repository and Overleaf thesis skeleton.	Completed
Design	Architected simulator–diffing–analysis pipeline.	Completed
Research	Finalized selection of LLM APIs and SAST tools.	Completed

Table 4: Finished Tasks and Milestones

6.2 Worked Hours

The following is copied from time warrior, a terminal based time utility

Date	Day	Tags	Start	End	Time	Total
2026-01-12	Mon	5 final-project, lecture	15:30:00	16:30:00	1:00:00	1:00:00
2026-01-17	Sat	4 final-project, reading	12:00:00	16:00:00	4:00:00	4:00:00
2026-01-18	Sun	3 final-project, reading	15:00:00	18:30:00	3:30:00	3:30:00
2026-01-19	Mon	2 final-project, lecture	15:30:00	16:30:00	1:00:00	1:00:00
2026-01-20	Tue	1 final-project, planning	11:00:00	16:00:00	5:00:00	5:00:00
2026-01-21	Wed	0 final-project, meeting	16:00:00	16:30:00	0:30:00	0:30:00
2026-01-26	Mon	final-project, planning	8:30:00	12:00:00	3:30:00	
		final-project, lecture	15:30:00	16:30:00	1:00:00	4:30:00
2026-01-30	Fri	final-project, meeting	13:00:00	13:30:00	0:30:00	0:30:00
2026-02-02	Mon	final-project, lecture	15:30:00	16:30:00	1:00:00	1:00:00
2026-02-03	Tue	final-project	16:18:18	22:28:17	6:09:59	6:09:59
2026-02-04	Wed	final-project	14:37:41	0:00:00	9:22:19	9:22:19
2026-02-05	Thu	final-project, writing	20:00:00	23:00:00	3:00:00	3:00:00
2026-02-07	Sat	final-project, writing	19:00:00	22:00:00	3:00:00	3:00:00
2026-02-08	Sun	final-project, writing	17:00:00	22:30:00	5:30:00	5:30:00

Total: 48:02:18

Table 5: Current Project Backlog

ID	Phase	Description	Priority
1	Phase I	Planning: Create product backlog and sprint structure, (kanban, github projects).	Medium
2	Phase I	Research: Deep dive into SilentSync, Col-orama, and related tooling.	Medium
3	Phase I	Thesis: Write project description and objectives.	Medium
4	Phase I	Thesis: Draft proper background section.	Medium

Table 5: Current Project Backlog

ID	Phase	Description	Priority
7	Phase I	Presentation: Prepare slides for Sitrep 1.	High
5	Phase I	Prototyping: Connecting Python to LLM APIs.	Medium
8	Phase I	Meeting: Attend Sitrep 1.	High
6	Phase I	Environment: Configure Python 3.9+ development environment.	Low
9	Phase I	Thesis: Complete first draft of introduction.	Medium
10	Phase II	Development: Implement PyPI simulator core.	High
11	Phase II	Development: Implement local storage/DB for packages.	Medium
12	Phase II	Thesis: Document simulator architecture.	Low
13	Phase II	Development: Implement diffing engine (Version A vs B).	High
14	Phase II	Testing: Unit test diffing logic.	Medium
16	Phase II	Development: Build pipeline controller (orchestration).	High
17	Phase II	Development: Implement logging system.	Medium

Table 5: Current Project Backlog

ID	Phase	Description	Priority
18	Phase II	Presentation: Prepare slides for Sitrep 2.	High
19	Phase II	Meeting: Attend Sitrep 2 (demo prototype).	High
20	Phase II	Development: Refine simulator based on initial tests.	Medium
21	Phase II	Thesis: Update implementation chapter.	Medium
22	Phase III	Development: Integrate Bandit and Semgrep.	High
23	Phase III	Configuration: Configure SAST rulesets for supply chain.	Medium
24	Phase III	Development: Integrate Gemini/GPT APIs.	High
25	Phase III	AI: Develop system prompts for detection.	High
26	Phase III	Dataset: Generate malicious samples (SilentSync, Colorama, etc.).	High
27	Phase III	Dataset: Select benign control versions.	Medium
28	Phase III	Thesis: Write experimental setup section.	Medium
29	Phase III	Integration: Connect Simulator $\rightarrow$ Diff $\rightarrow$ Analysis $\rightarrow$ DB.	High
30	Phase III	Testing: Perform dry run of full system.	Medium

Table 5: Current Project Backlog

ID	Phase	Description	Priority
31	Phase III	Execution: Run automated benchmarking suite.	Medium
33	Phase III	Presentation: Draft final defense slides.	High
34	Phase IV	Meeting: Attend Sitrep 3 (Generalprufa).	High
35	Phase IV	Analysis: Compare detection rates AI vs SAST.	High
36	Phase IV	Thesis: Write results and evaluation chapters.	Medium
37	Phase IV	Thesis: Write discussion and conclusion.	High
38	Phase IV	Documentation: Write README and user manuals.	Medium
39	Phase IV	Review: Full thesis read-through.	Medium
40	Phase IV	Code: Final polish and cleanup for submission.	High
41	Phase IV	Submission: Submit thesis to Skemman.	High
42	Phase IV	Defense: Final public presentation.	High

References

- [1] “PyPI · The Python Package Index,” PyPI, Accessed: Feb. 8, 2026. [Online]. Available: <https://pypi.org/>.
- [2] M. U. Zeshan et al., “Many Hands Make Light Work: An LLM-based Multi-Agent System for Detecting Malicious PyPI Packages,” *Journal of Systems and Software*, Jan. 23, 2026.
- [3] J. Mahon, C. Hou, and Z. Yao. “PyPitfall: Dependency Chaos and Software Supply Chain Vulnerabilities in Python.” arXiv: [2507.18075 \[cs\]](https://arxiv.org/abs/2507.18075), Accessed: Jan. 22, 2026. [Online]. Available: <http://arxiv.org/abs/2507.18075>, pre-published.
- [4] W. Guo et al., “Cutting the Gordian Knot: Detecting Malicious PyPI Packages via a Knowledge-Mining Framework,” *arXiv preprint arXiv:2601.16463*, Jan. 23, 2026.
- [5] D. L. Vu, I. Pashchenko, F. Massacci, H. Plate, and A. Sabetta, “Towards Using Source Code Repositories to Identify Software Supply Chain Attacks,” in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’20, New York, NY, USA: Association for Computing Machinery, Nov. 2, 2020, pp. 2093–2095, ISBN: 978-1-4503-7089-9. DOI: [10.1145/3372297.3420015](https://doi.org/10.1145/3372297.3420015). Accessed: Jan. 22, 2026. [Online]. Available: <https://dl.acm.org/doi/10.1145/3372297.3420015>.